

Distributed Ledger Technology

Blockchain as Middleware

Distributed Systems / Case Study

Andrea Omicini Giovanni Ciatto

<mailto:andrea.omicini@unibo.it> <mailto:giovanni.ciatto@unibo.it>

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



Outline of Part I. Middleware & DLT

- 1 Middleware
- 2 Distributed Ledger Technology (DLT)



Outline of Part II. Blockchain

- 3 Expectations
- 4 Background
- 5 State Machine Replication
- 6 Blockchain Main Elements
- 7 Universal SMR



Outline of Part III. Smart Contracts

- 8 Smart Contracts
- 9 Examples & Use Cases
- 10 Issues



Part I

Middleware & DLT



Next in Line...

- 1 Middleware
- 2 Distributed Ledger Technology (DLT)

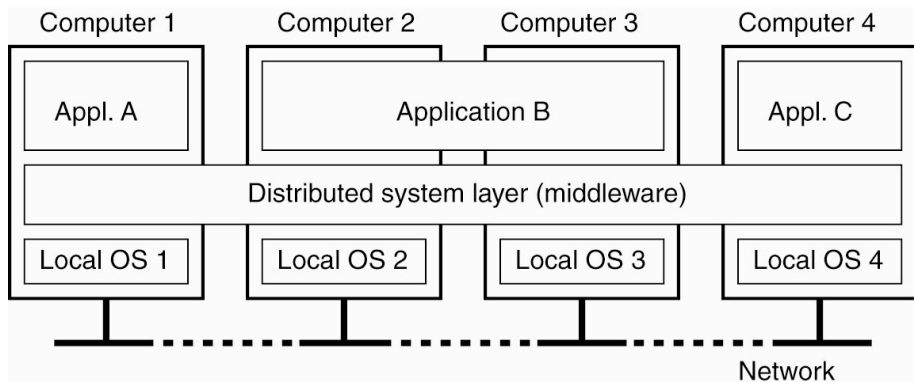


What is Middleware?

Traditional definition

- what is **middleware**?
 - the word suggests something belonging to the middle
 - but middle between what?
- the traditional middleware definition
the middleware lies in the middle between the operating system and the applications
- the traditional definition stresses *vertical* layers
 - applications on top of middleware on top of the OS
 - middleware-to-application interfaces (**top interfaces**)
 - middleware-to-OS interfaces (**bottom interfaces**)

The Typical Depiction of Middleware



[Tanenbaum and van Steen, 2017]

Why Middleware?

Behind middleware

- problems of today
 - software development is *hard*
 - experienced designers are *rare* (and *costly*)
 - applications become more and more complex
- what can middleware help with?
 - middleware is *developed once* for *many applications*
 - higher-quality *designers* can be afforded
 - middleware can provide *services* to applications
 - middleware *abstracts* away from the *specific OS*

Middleware and Models I

Interoperability

- a key feature of middleware is **interoperability**
 - applications using the same middleware can interoperate
 - this is true of any common platform (e.g. OS file system)
- however, many incompatible middleware systems exist
 - applications on middleware *A* can work together
 - applications on middleware *B* can work together, too
 - but, *A*-applications and *B*-applications most often cannot
- the *Enterprise Application Integration* (EAI) task
 - emphasis on *horizontal* communication
 - *application-to-application* and *middleware-to-middleware*

Middleware and Models II

Conceptual integrity

- software development does not happen *in vacuum*
 - almost any software project must cope with past systems
 - there is never time nor resources to start *from scratch*
 - legacy systems were built with their own approaches
- system integration is the only way out
 - take what is already there and add features to it
 - try to add without modifying existing subsystem
- first casualty: **conceptual integrity**
 - the property of a system of being understandable and explainable through a coherent, limited set of concepts

Middleware and Models III

Models from middleware to applications

- real systems are heterogeneous
 - piecemeal growth is a *very troublesome* path for software evolution
 - still, it is very popular – being asymptotically the most cost effective when development time goes to zero
- middleware technology is an *integration* technology
 - adopting a given middleware should ease *both* new application development *and* legacy integration
 - to achieve integration while limiting conceptual drift, middleware tries to cast a **model** on heterogeneous applications.

Middleware Technologies

Abstract vs. concrete middleware

- **abstract middleware**: a common *model*
- **concrete middleware**: a common *infrastructure*
- example: *distributed objects*
 - abstractly, any middleware modelling distributed systems as a collection of network reachable objects has the same model: OMG CORBA, Java RMI, MS DCOM, OSGI Architecture. . .
 - actually, even at the abstract level there are differences. . .
 - concrete implementations, instead, aim at actual interoperability, so they must handle much finer details
 - until CORBA 2.0, two CORBA implementations from different vendors were not interoperable
 - a number of OSGI implementations nowadays available: Eclipse Equinox, Apache Felix, Knopflerfish, . . .

Middleware Standards

The role of standards

- dealing with infrastructure, a key-issue is the so-called *network effect*
 - the value of a technology grows with the number of its adopters
- **standardisation efforts** become critical to build momentum around an infrastructure technology
 - large standard consortia are built, which gather several industries together

OMG CORBA <http://www.omg.org/spec/CORBA/>

FIPA <http://www.fipa.org/specifications/>

OSGi <http://www.osgi.org/developer/specifications/>

W3C <http://www.w3.org/standards/>

- big industry players try to push their technology as *de facto* standards, or set up more open processes for them

What Kind of Middleware Would We Like Here & Now?

Which hot features would we need now?

- ook authentication, authorisation, security, distributed access to shared resources, . . .
- ? what about *consensus*, and consensus algorithms?
- should we write them from scratch for every distributed application of ours that needs them?
- or, should we be provided with them as part of new *hot middleware*, too?

Next in Line...

- 1 Middleware
- 2 Distributed Ledger Technology (DLT)



Definitions I

DLT [Focus Group on Application of Distributed Ledger Technology, 2017]

Distributed ledger technology (DLT) refers the processes and related technologies that enable *nodes in a network* to securely propose, validate, and record *state changes* (or updates) to a **synchronised ledger** that is *distributed* across the network's nodes

Where is synchronisation?

- the ledger is synchronised
 - whereas the nodes and the network are *not* required to be synchronous
- ⇒ so, the DLT is expected to work as a *synchronising layer* for the processes
- as middleware can be expected to do in a distributed system

Definitions II

DLT^[ISO/TC 307, 2024]

ledger — information store that keeps records of transactions that are intended to be *final*, *definitive*, and *immutable*

distributed ledger — *ledger* that is shared across a set of DLT nodes, and *synchronised* between the DLT nodes using a *consensus mechanism*

distributed ledger technology — technology that enables the operation and use of distributed ledgers

Standards I

The role of standardisation

- distributed systems are typically required to be *open*—not just secure, reliable, available, ...
- shared **standards** works at the core of any open system
- *middleware* is the most obvious place for *standards*
- *standardising middleware* works as the most general foundation possible to that



Standards II

Standardisation efforts for DLT

ITU the International Telecommunication Union (ITU) is the United Nations specialised agency for information and communication technologies—ICTs

<https://www.itu.int/>

ITU FG DLT Focus Group on Application of Distributed Ledger Technology

<https://www.itu.int/en/ITU-T/focusgroups/dlt/>

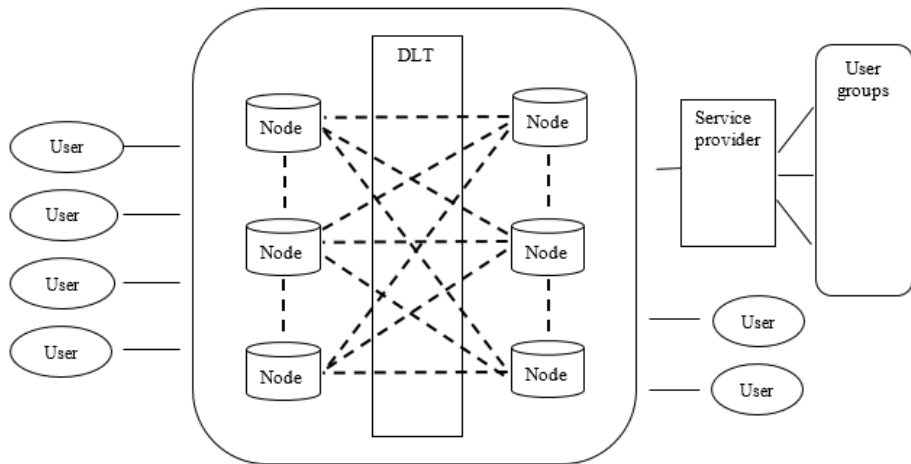
ISO the International Organization for Standardization is a worldwide federation of national standards bodies

<https://www.iso.org/>

ISO/TC 307 ISO Technical Committee on blockchain and distributed ledger technology

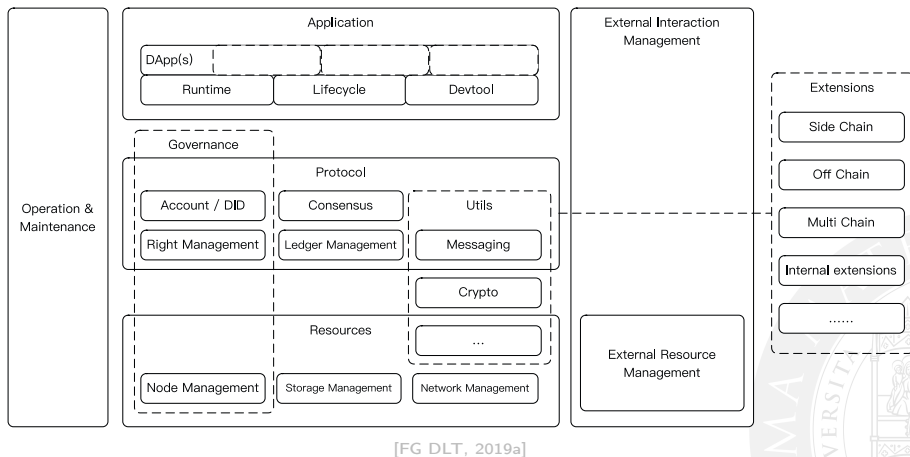
<https://www.iso.org/committee/6266604.html>

DLT Actors & Components



[FG DLT, 2019b]

DLT Architecture I



DLT Architecture II

[FG DLT, 2019a]

The high-level architecture constrains the highly abstract hierarchical architecture of distributed ledgers. The high-level architecture can cover almost all distributed ledgers, including public chains represented by Ethereum^[Wood, 2014] and Bitcoin^[Nakamoto, 2008], private chains represented by Hyperledger Fabric^[Androulaki et al., 2018] and non-blockchain distributed ledgers systems.



DLT Use Cases

Some reported use cases^[FG DLT, 2019c]

- finance** digital letter of credit, blockchain for insurance
- pharma** blockchain web/mobile application for vaccine supply, drugs distribution
- medicine** blockchain for bone marrow, blood, and organ, health data records
- ICT** global market place for mobile operators and service providers, DLT for number assignment, services and number portability
- energy** energy distribution with the use of smart contracts, P2P energy trading

... ..

DLT Benefits I

General benefits^[FG DLT, 2019c]

- DLT is a *secure*, cost-effective technology that enables the deployment of **globally-scalable services**
- DLT is considered as a *tamper-resistant* and *auditable* technology that is resistant to **systemic failures**
- DLT is an effective tool to *detect* and mitigate **fraud**
- as a *middleware* instance, DLT can be seen as a form of General Purpose Technology (GPT)
 - it does not stand for itself, instead it also brings gains to *other technologies* and sectors
 - so it may take a long time to reach mass adoption, but once adopted it leads to *productivity gains* across multiple industries

DLT Benefits II

Specific benefits^[FG DLT, 2019c]

transparency & trust DLT is perceived as a canonical trusted and transparent shared resource that makes interactions and transactions *understandable*, *traceable*, *certifiable*, and *accountable*—particularly relevant for interactions among untrusted users

security encryption of data, access control, tamper-resistant data, identity management, fault tolerance

incentives *cost reduction* through *disintermediation* and increased efficiency

... ..

DLT Journal

ACM DLT

- “Distributed Ledger Technologies: Research and Practice”
- <https://dl.acm.org/journal/dlt>

Distributed Ledger Technologies: Research and Practice (DLT) is a peer-reviewed journal that seeks to publish high quality, interdisciplinary research on the research and development, real-world deployment, and/or evaluation of distributed ledger technologies (DLT); e.g., blockchain, cryptocurrency, and smart contract. DLT will offer a blend of original research work and innovative practice-driven advancements by internationally distinguished DLT experts and researchers from academia, and public and private sector organizations.

DLT vs. BCT

DLT = BCT?

- blockchain technology (BCT) clearly looks like DLT
- however, a blockchain is a way to implement a distributed ledger, but not all distributed ledgers necessarily employ blockchains
- ! so, in general, $DLT \supset BCT$
- e.g.
 - IOTA <https://iota.org>
 - Hedera Hashgraph <https://hedera.com>
 - CORDA <https://corda.net>

Part II

Blockchain



Next in Line...

3 Expectations

4 Background

5 State Machine Replication

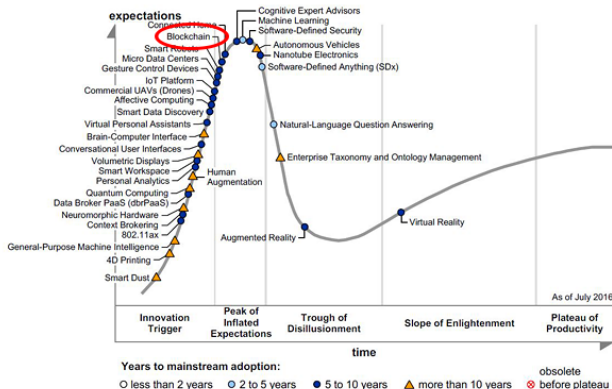
6 Blockchain Main Elements

7 Universal SMR



DLT or Blockchain? Expectations I

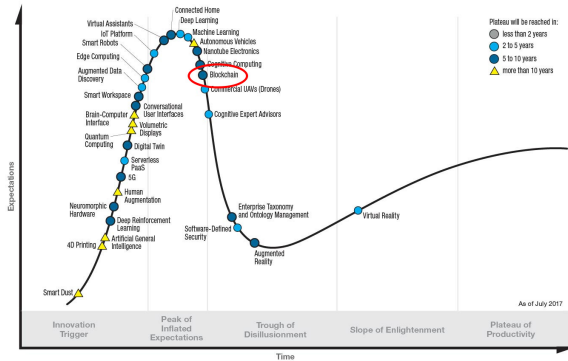
Gartner Hype Cycle for Emerging Technologies, 2016



Source: Gartner (July 2016)

DLT or Blockchain? Expectations II

Gartner Hype Cycle for Emerging Technologies, 2017



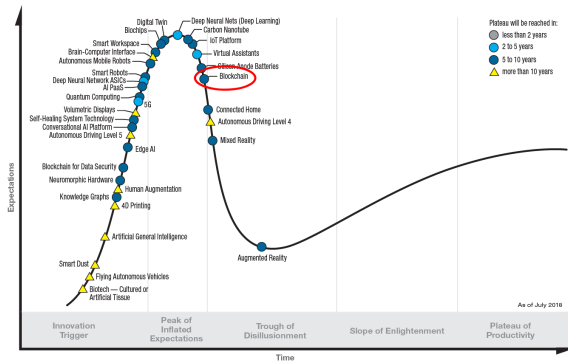
gartner.com/SmarterWithGartner

Source: Gartner (July 2017)
© 2017 Gartner, Inc. and/or its affiliates. All rights reserved.

Gartner

DLT or Blockchain? Expectations III

Hype Cycle for Emerging Technologies, 2018



gartner.com/SmarterWithGartner

Source: Gartner (August 2018)
© 2018 Gartner, Inc. and/or its affiliates. All rights reserved.

Gartner.

DLT or Blockchain? Expectations IV

... what happens after 2018?

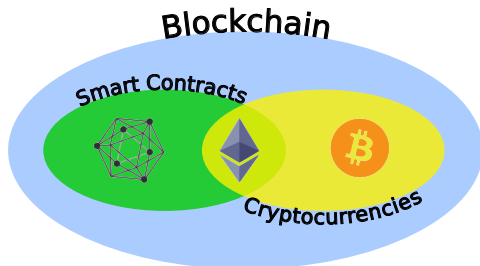
- no more blockchain in the Gartner Hype Cycle for Emerging Technologies
- ! ITU (2019), ^[FG DLT, 2019b] ISO (2020) ^[ISO/TC 307, 2020] bring to a successful conclusion their standardisation efforts
- possibly meaning that DLT / BCT are going fast forward towards maturity and widespread industrial exploitation



Blockchain as the Bitcoin Technology? I

Blockchain $\neq$ Bitcoin

blockchain $\supset$ cryptocurrencies $\supset$ Bitcoin



HyperLedger Fabric



Ethereum



Bitcoin

Blockchain as the Bitcoin Technology? II

The blockchain is ...

... a model for *distributed & decentralised computation*

- can be **implemented** in diverse ways
- within heterogeneous **application domains**
- and with disparate goals

Bitcoin^[Nakamoto, 2008] is ...

... one specific blockchain implementation

- within a specific application domain (**cryptocurrencies**)
- with a specific goal (**digitalise & decentralise fiat currency**)

Next in Line...

3 Expectations

4 **Background**

5 State Machine Replication

6 Blockchain Main Elements

7 Universal SMR



The Blockchain at a Glance

Blockchain

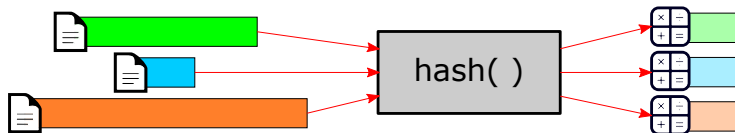
- a shared, transparent, distributed, *append-only* ledger
 - i.e., a register tracking the *order & timing* of transactions
- secured using cryptographic schemes
 - e.g., 1-way hash functions, digital signatures and certificates, encryption
- ledger replicated on multiple nodes of a peer-to-peer network
 - avoiding the need for a *trusted* centralised intermediary
 - making the ledger immutable
- transactions are approved and propagated through consensus
 - preventing data loss/corruption due to machines crashing/lying
 - i.e., providing *robustness* against Byzantine failures
- ideal use case
 - secure hub for *mutually-untrusted* parties who need to interact

Focus on...

- 3 Expectations
- 4 Background
 - **Cryptographic Tools**
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



One-way Hash Functions



Properties

! ✓ = easy, × = infeasible

- $digest = H(input)$
 - ✓ $input \rightarrow digest$
 - × $digest \rightarrow input$
 - ! $\mathbb{P}(H(x) = H(y) \mid x \neq y) \approx 0$
 - $small \Delta input \implies big \Delta digest$
- goal: revealing **tampering** attempts

Example: the SHA512 hash function

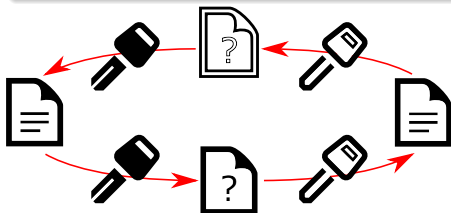
```
hash("Alice->Bob: 1 $"): "b35f8926bc17290dc4f4febb98bd1..."
hash("Allce->Bob: 1 $"): "ff65fc2d9d8cf76917d47b0c7a0a7..."
hash("Alice->bob: 2 $"): "7042a70cc7ec19c5cc17bf89390d1..."
```


Public-Key (Asymmetric) Cryptography

User holds two keys

public key is publicly available

private key is kept secret



Properties

! ✓ = easy, × = infeasible

- ✓ creating key-couple
- ✓ encrypting with one key
- ✓ decrypting with the other key
- × decrypting with wrong key
- × deriving one key from the other

goals: confidentiality, authenticity, non-repudiation

Use case

- *public-key authentication*

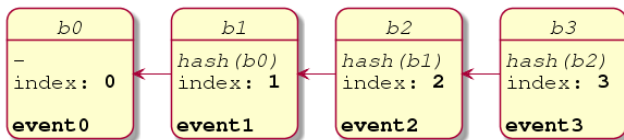
Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - **Schemes**
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR

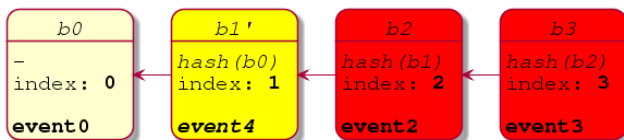


Hash Chains

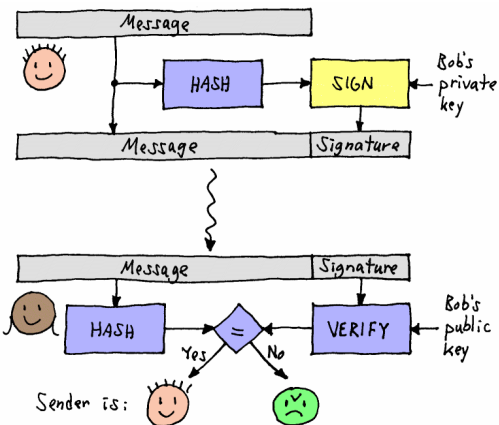
Hash functions can be used to build **hash chains**



- used to keep track of **sequences** of events—their *relative order*
- each $event_i$ is stored within a block b_i
- each block b_{i+1} records the hash of the previous block $H(b_i)$
 - so to prevent attempts to alter / tamper with the sequence



Digital Signatures



- senders attach **signatures** to their messages
- $Signature = \text{Encrypt}_{Priv}(H(Message))$
- signatures can be verified by using *Pub*

Authenticity

alleged sender $\equiv$ actual sender

Integrity

message was not tampered

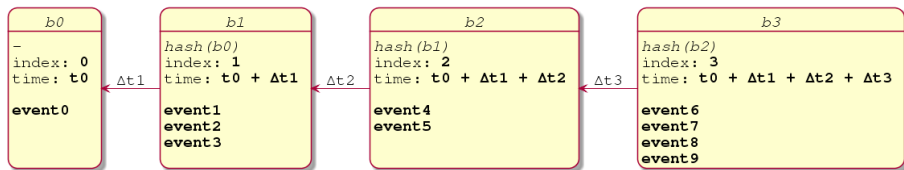
Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - **Applications**
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



Timestamping Service

By including timestamps in hash chains, we get a **timestamping** service



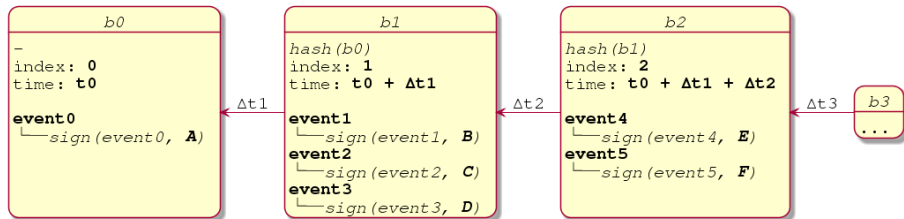
Properties

- blocks are assumed to be appended (almost) **periodically**
- several **simultaneous** events may be stored within the same block
- keeps track of **what** happened and **when**

Notary Service – Basic Idea

By making timestamping services only accept correctly-**signed** events, we get a **notary service**

! below, letters A, . . . , F represent user **identities**



Properties

- signatures must be **valid**—otherwise, events cannot be **notarised**
- keeps track of **who** did **what** and **when**
- provides **non-repudiation** of events and their timing

Notary Service – Applications

A notary service may have several real-world useful applications

- a shared ledger keeping track of *orders, shipments, and payments* between companies
 - e.g. providers and suppliers in a supply chain
- a voting system (with *open voting*) keeping track of who already voted which option
- an open registry keeping track of *public calls* and their applications
- a registry for *building ownerships*, keeping track of property transfer
- a *punching in/out* system for employees
- a cryptocurrency

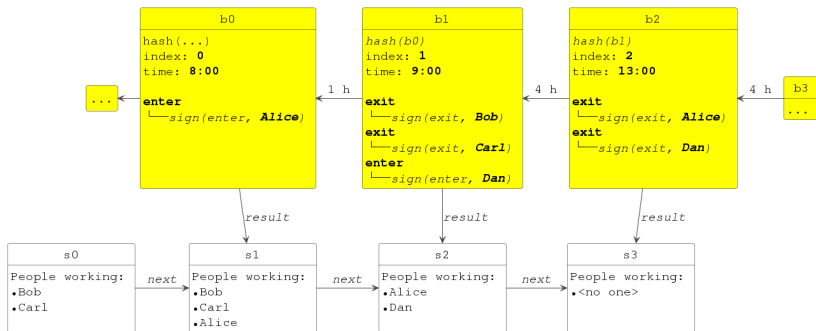
In general. . .

. . . any use case where **two or more** parties may have interest in **lying** about **what** they did or **when** could take advantage of a notary service

Notary Service – Example

A punching in/out notary service for employees

- each employee can either **enter** or **exit** at a given time



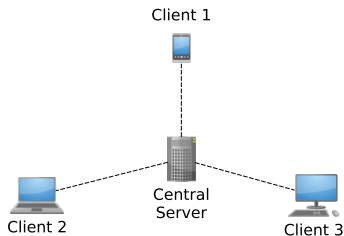
Resulting state

Notice that, after each block, a **resulting state** can be computed taking into account all events occurred so far

! white boxes in the image

Notary Service – Architecture

In practice, a notary service can be **implemented** as a distributed system having a **logically-centralised** architecture



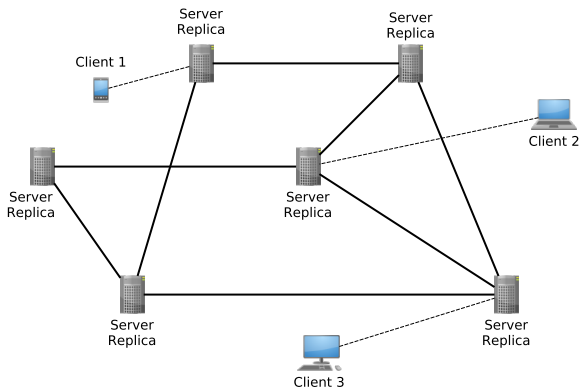
- the central server is in charge of
 - verifying events validity
 - maintaining the hash chain
 - storing the events data
- it may be redundant but it is **controlled** by a single party/organization
- clients send new events

Single point of *trust*

The party/organisation running the server may be(come) **corrupted** and arbitrarily **alter** the ordering/timings of events, or **prevent** a particular system from participating. *No defence against this problem.*

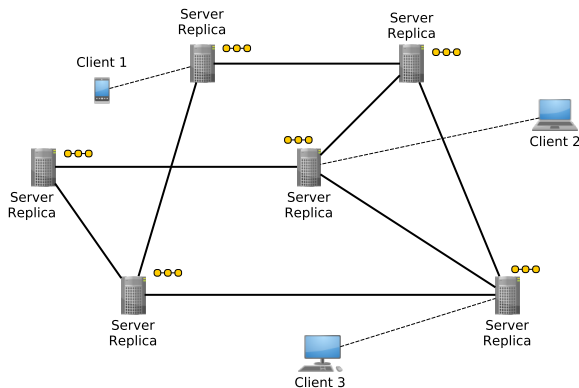
The Need for Replication I

- 1 in order to avoid *centralisation of trust*, we may **replicate** the notary service in such a way that each **replica** is controlled by a different party/organisation



The Need for Replication II

- 2 whenever a client sends a new event to a replica, is it **propagated** to all the other replicas
- 3 each replica **stores** a copy of the hash chain and the events



The Need for Replication III

- ④ it is still possible for a replica to be compromised/corrupted, but as long as the **most of them** are not, attempts to tamper can be detected
 - “most”? how many?
- ⑤ essentially, we are proposing to replicate the **software** validating the many events sent by users

State Machine Replication^[Schneider, 1990, Charron-Bost et al., 2010]

- this is the aim of the **State Machine Replication** approach

Next in Line...

3 Expectations

4 Background

5 State Machine Replication

6 Blockchain Main Elements

7 Universal SMR



Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - **Overview**
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



State Machine Replication (SMR)

Main idea^[Schneider, 1990, Charron-Bost et al., 2010]

Executing the **same** (not necessarily finite) **state machine** over multiple independent (possibly distributed) **processors**, in parallel, in order to achieve

- fault tolerance—to stops, crashes, lies, bugs, etc.
- availability and reactivity
- data / software replication & untamperability

A network of replicas

The processors constitute a **peer-to-peer** (P2P) network of **replicas**, all exposing the same **observable behaviour**

! no assumption about the topology

Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - **Deterministic Stateful Computation**
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



Stateless vs. Stateful Computation

$Input \longrightarrow Computation \longrightarrow Output$

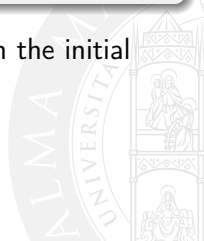
Computation is *stateless* when *output* depends on *input* only

- e.g., *functions*

$(Input, State) \longrightarrow Computation \longrightarrow (Output, State')$

Computation is *stateful* when *output* depends on both *input* and *state*

- or, in other terms, output at every step depends on both the initial state and the previous *sequence of inputs*
- e.g., *objects*
- ! objects have *memory*, functions have not



Deterministic vs. Non-deterministic Computation

Deterministic model of computation

A computational machine is deterministic if its state depends only on the initial state and the sequence of inputs

Potential sources of non-determinism

- *randomiser* function
- concurrency

Non-deterministic system behaviour with deterministic computation

- the issue of *observation*: where do we set the boundary?
- the role of *input*:^[Wegner and Goldin, 1999] the behaviour of the system composed by the computational machinery and its surrounding environment is overall non-deterministic

Deterministic Stateful Computation

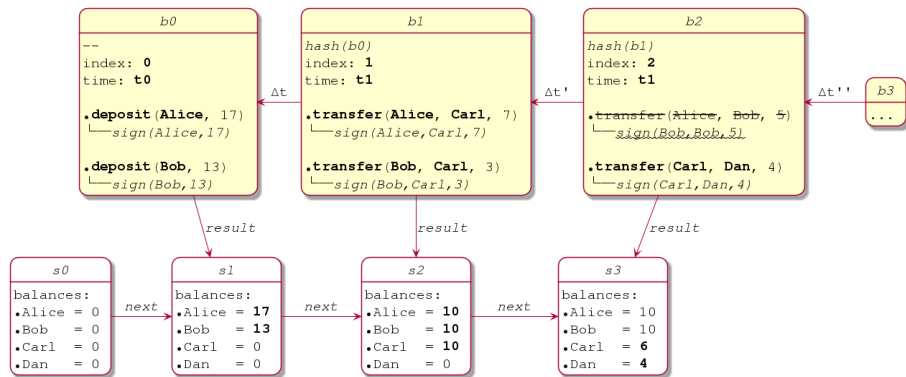
Deterministic stateful machines. . .

- can be arbitrarily **replicated**
- replicas can be run **concurrently**
- in order to be arbitrarily replicated & executed concurrently
 - every replica should start from the same **initial state**
 - every replica should process *inputs* in the **same order**
- as a result, they all would
 - move through the same **succession** of states
 - maintain the **consistency** of the state w.r.t. inputs



Example: Simple Cryptocurrency

notary service + replication + Ledger class $\approx$ cryptocurrency



Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - **Issues of Open Distributed Systems**
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



SMR & Open Distributed Systems: Challenges

SMR deals with open & distributed systems—which lead to some challenges

- messages may be lost, corrupted, reordered, or duplicated while passing through the network
 - ⇒ each replica may *perceive* a different view on the system events
 - ? how to enforce the same ordering of events for all replicas?
- lack of *global time* across distributed machines
 - it is difficult to keep clocks *synchronised* among replicas
 - ⇒ no trivial solution for events ordering (e.g. *timestamps*)
 - ? how to share a common time?
- CAP theorem^[Brewer, 2000]
 - ? what if replicas cannot (temporarily) communicate with each other?
- byzantine generals problem^[Lamport et al., 1982] and byzantine faults
 - ? what if some replica crashes or *misbehaves* (because is buggy or corrupted)?

SMR & Open Distributed Systems: Responses

SMR deals with open & distributed systems—challenges may find some responses

- messages may be lost, corrupted, reordered, or duplicated while passing through the network
 - ✓ robust transport protocols such as TCP
 - ✓ hash functions for integrity
- lack of global time across distributed machines
 - well-known & well-studied problem in the literature
 - ✓ reasonable to assume machines have access to **UTC** (Universal Coordinated Time) through **NTP** (Network Time Protocol)
- CAP theorem
 - ~ eventual consistency
- byzantine generals problem and byzantine faults
 - ~ redundancy + byzantine fault-tolerant **consensus** algorithms



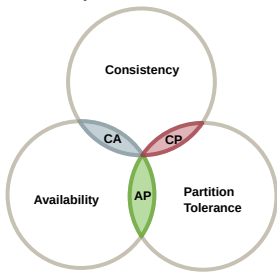
Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - **CAP Theorem**
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



CAP Theorem: Fast Recap ^[Brewer, 2000]

In distributed systems with several machines serving some shared data to multiple clients



Consistency (C): all clients always...

... have the same view on data, even after updates

Availability (A): all clients always...

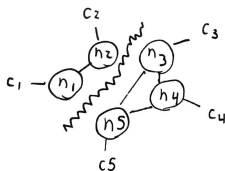
... receive some timely response, even with failures

Partition-tolerance (P): the system...

... keeps operating regardless of network partitions

Theorem: no more than two of them at a time

E.g., if you want P, you must choose between A and C



Eventual Consistency

Eventual Consistency

Sooner or later, in **finite time**, things will become consistent

- where “sooner or later” $\approx$ “in reasonable time”
- the blockchain basically chooses **P** and **A**
- similarly to distributed databases (e.g., MongoDB, or noSQL in general), it provides for **eventual consistency**
 - often, consistency is governed by **probabilistic** rules



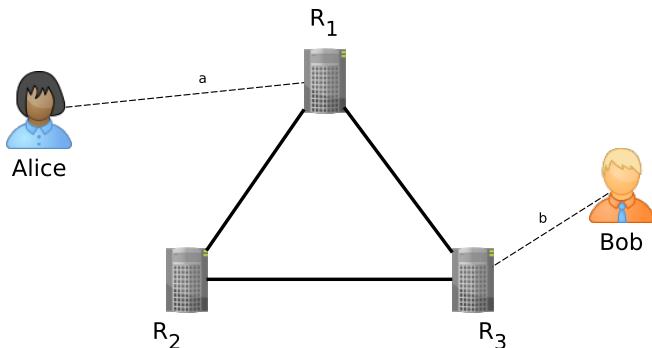
Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - **Byzantine Fault Tolerance**
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



Byzantine Faults I

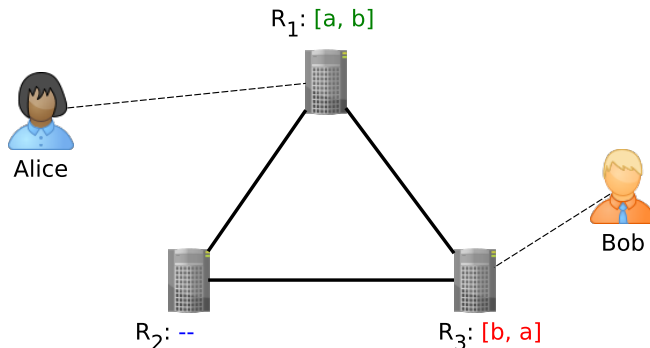
- 1 Alice and Bob, **simultaneously** try to update a replicated system by contacting a replica



- 2 the final result depends on the requests **ordering**

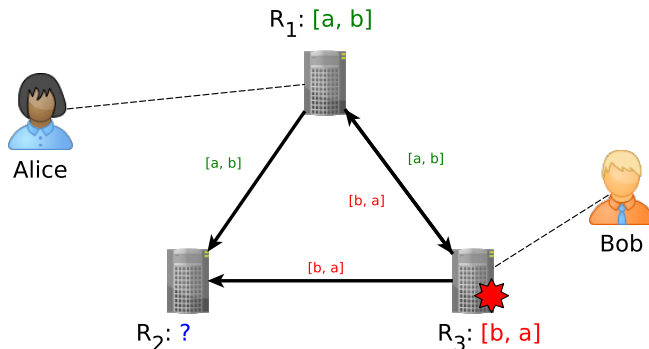
Byzantine Faults II

- 3 all replicas must **agree** on an **ordering** for the operations to be committed



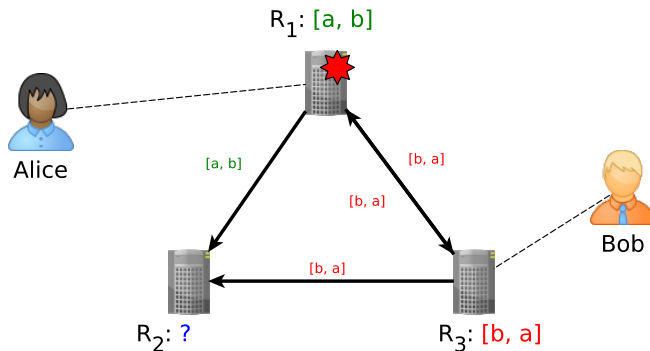
Byzantine Faults III

- ④ byzantine faults (i): **inconsistencies** may arise & prevent agreements



Byzantine Faults IV

- 5 byzantine faults (ii): nodes may be deliberately lying



- 6 some 4th replica may break the **impasse**...

Consensus I

- due to the **value** possibly stored in the ledger, bad actors may have economic **incentives** to try and cause faults
 - e.g., money or advantages
- in absence of **Byzantine Fault Tolerance** (BFT), a replica may *alter* the blocks timing or ordering, or excluding particular events
- the perspective of a replica producing Byzantine faults is dreadful w.r.t. *consistency*: it may indefinitely prevent agreements
- blockchain, similarly to distributed databases, usually exploits **BFT consensus algorithms** in order to prevent Byzantine failures



Consensus II

BFT consensus algorithm

A distributed protocol aimed at making a SMR system resistant (i.e., *ensuring eventual consistency*) to Byzantine failures

Consensus impossibility

It turns out *distributed consensus is impossible* under *any* of the following assumptions

- 1 the underlying network is *asynchronous*,^[Fischer et al., 1985] i.e., if messages can be arbitrarily **re-ordered** or **delayed**
 - this is why all consensus protocols proceed through **periodic rounds**
- 2 the amount f of *Byzantine-faulty* replicas is such that $f \geq \frac{1}{3}N$, being N the total amount of replicas^[Lamport et al., 1982]
 - this is why we needed the **4th replica**

Next in Line...

- 3 Expectations
- 4 Background
- 5 State Machine Replication
- 6 Blockchain Main Elements**
- 7 Universal SMR



Disclaimer

- most blockchain-related works describe *specific* blockchain technologies (**BCT** henceforth) using a bottom-up approach
- this hinders *generality*, and limits the discussion the overall *potential of BCT*
- in the following, we adopt a *top-down* approach to explaining the blockchain, drawing from a number of prominent sources^[Nakamoto, 2008, Wood, 2014, Androulaki et al., 2018]
- as usual, errors and problems of any sort depend on the authors of these slides
- the following description of the blockchain architecture and functioning is strongly inspired to Ethereum^[Wood, 2014]
 - as the most mature, studied, documented smart-contract-enabled BCT

Blockchain Overview

Generally speaking, a *blockchain technology* (BCT) is ...

A clever implementation of a SMR system keeping track of which **users** own some **assets** (representations), by means of a *replicated ledger*, updated through **consensus**

users — are identified through **public keys** and must sign their **transactions** with them

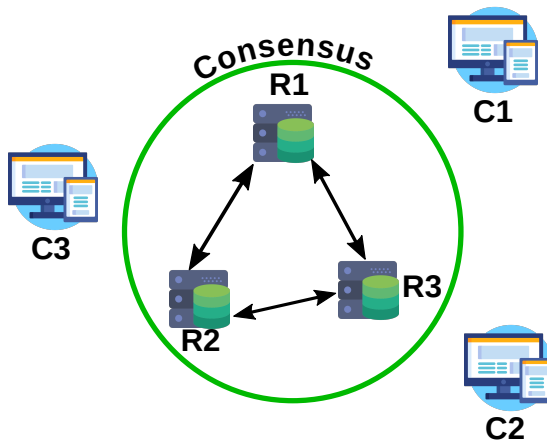
assets — may be anything having some **value**, an **owner**, and, possibly, some mutable **state** to be tracked

- e.g., events, money, documents, notary acts, property acts, contracts, etc.

ledger — is a replicated notary service holding assets' property/state evolution

consensus — a distributed protocol aimed at maintaining and updating the ledger replicas in sync

The Architecture of a BCT



- replicas $R_1, \dots, R_N$ actually **store** a copy of the *whole ledger*
- clients $C_1, \dots, C_M$ may **propose** some asset **updates** to any R_i
 - they have no need to store the whole ledger

Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - **User Identifiers**
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



User Identifiers

Users

- they are supposed to **own** (at least) one (K_{pub}, K_{pr}) key pair
- they are **identified** by some function $f(K_{pub})$ of their public key
 - e.g. 1-way-hash functions
 - e.g. digital certificates issued by some **trusted CA**

! identifiers are also known as **addresses** in this context

Permissioned vs. Permissionless

- each user either owns *multiple non-intelligible identifiers*...
 - ✓ pseudonymity
 - ✓ decentralised
 - ✗ Sybil attack^[Douceur, 2002]
- ...or he/she *owns a single certified identifier*
 - ✗ single point of failure/trust

Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - **The World State**
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



The World State I

System state

$$\begin{aligned}\langle \text{SystemState} \rangle &::= \langle \text{Asset} \rangle \\ &\quad | \quad \langle \text{SystemState} \rangle \cup \langle \text{SystemState} \rangle \\ \langle \text{Asset} \rangle &::= \langle \text{UserID} \rangle, \langle \text{AssetState} \rangle\end{aligned}$$

The system state **conceptually** consists of a **set** of asset representations, each one specifying the owner of the asset

where

- $\langle \text{UserID} \rangle = f(K_{pub}^U)$, and K_{pub}^U is the public key of some user U

The World State II

Asset state

e.g. $\langle \text{AssetState} \rangle ::= \langle \text{Balance} \rangle$

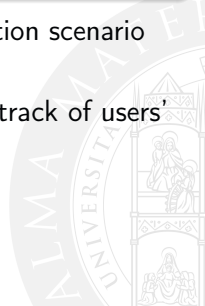
e.g. $\langle \text{AssetState} \rangle ::= \langle \text{CustomData} \rangle$

The asset state *conceptually* consists of several fields keeping track of what a particular entity currently owns

The actual fields may vary depending on the specific application scenario where the blockchain is used

e.g. BCT coming with native cryptocurrencies, usually keep track of users' **balances** (at least)

e.g. other BCT may let users keep track of **custom data**



The World State – Possible Implementations

- account-based
e.g. Ethereum^[Wood, 2014]
- Unspent Transaction Output (UTXO)
e.g. Bitcoin^[Nakamoto, 2008]
- versioned key-value store
e.g. Hyperledger Fabric^[Androulaki et al., 2018]



Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - **Transactions**
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



Transactions—a.k.a. Inputs, Messages, TX I

Transaction

$$\langle TX \rangle ::= \langle UserID \rangle, \langle Operation \rangle, \langle Arguments \rangle, \langle Signature \rangle$$

Transactions represent (potential, proposed) **variations** of the system state

$\langle UserID \rangle$ — the address of the user issuing the TX, i.e., $f(K_{pub}^U)$

$\langle Operation \rangle$ — a description of the operation to be applied on the system state

$\langle Arguments \rangle$ — some arguments for the aforementioned operation

$\langle Signature \rangle$ — the cryptographic signature of the TX

Transactions—a.k.a. Inputs, Messages, TX II

The particular set of **possible operation** and the corresponding **transaction structures** is application-specific In the general case, you need at least:

- an operation **creating** new assets
- an operation **updating** existing assets

Example: *cryptocurrencies* transactions

! recall the Ledger script

$$\begin{aligned} \langle TX \rangle &::= \langle UserID \rangle, \text{deposit}, \langle Value \rangle, \langle Signature \rangle \\ &\quad | \quad \langle UserID \rangle, \text{transfer}, \langle UserID \rangle, \langle Value \rangle, \langle Signature \rangle \end{aligned}$$

where the two $\langle UserID \rangle$ fields represents the **sender** and **receiver**

Transactions—a.k.a. Inputs, Messages, TX III

Transactions validity

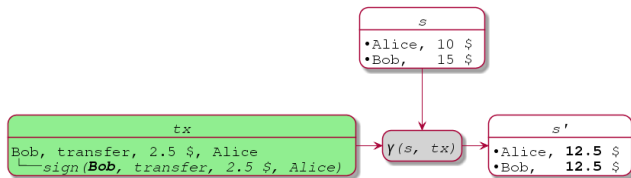
A transaction is **valid** if: (i) it is **well formed**, (ii) the **signature matches** the issuer address, and (iii) the signature certifies the transaction **integrity**

Transactions execution

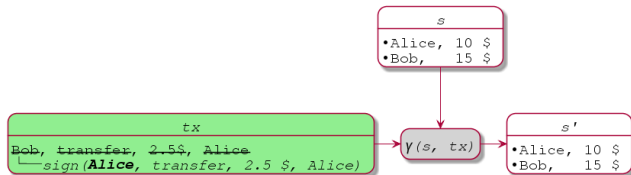
A *valid* transaction tx can be *applied* to a system state s by means of function γ producing a new system state s' , a.k.a. the *resulting* state of tx

$$\gamma(s, tx) = s'$$


Transactions—a.k.a. Inputs, Messages, TX IV



! valid transaction



! wrong signature

Transactions—a.k.a. Inputs, Messages, TX V

Depending on the specific use case at hand, it may happen that $s \equiv s'$

- for instance, because the sender has **not enough money** in s
- or because some other application-level **error** occurred

Function γ and SMR

In practice, function $\gamma(\cdot, \cdot)$ is the state machine (i.e., program) being replicated



Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - **Blocks & Block Chains**
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



Blocks & Block Chains I

Blocks

$$\langle Block \rangle ::= \langle PrevHash \rangle, \langle Height \rangle, \langle Time \rangle, \langle Metadata \rangle, \langle TxList \rangle$$
$$\begin{aligned} \langle TxList \rangle &::= \langle TX \rangle, \langle SystemState \rangle \\ &| \langle TxList \rangle, \langle TxList \rangle \end{aligned}$$

Blocks are **timestamped**, **hash-linked** lists of transactions and their corresponding resulting states

$\langle PrevHash \rangle$ — the hash of the previous block

$\langle Height \rangle$ — the index of the current block

$\langle Time \rangle$ — the timestamp of the current block

$\langle Metadata \rangle$ — some application-specific metadata

$\langle TxList \rangle$ — the list of transactions included into the current block and (the hashes of) their resulting states

Blocks & Block Chains II

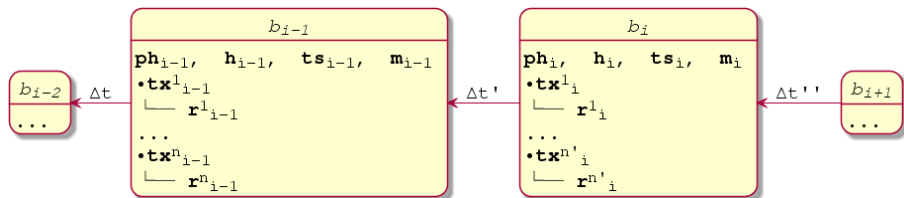
A block b_i is a 5-uple

$$b_i = (ph_i, h_i, ts_i, m_i, (tx_i^1, r_i^1, \dots, tx_i^{n_i}, r_i^{n_i}))$$

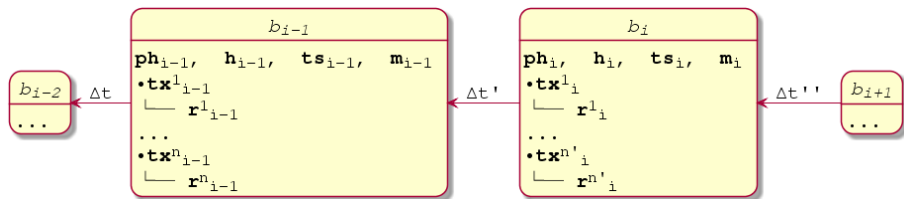
where

r_i^j is the resulting state after applying all transaction tx_i^k contained in b_i for $k = 1, \dots, j$

n_i is the total amount of transactions contained in b_i



Blocks & Block Chains III



Blocks local validity

Let b_{i-1} be the local replica's last known block, a novel block b_i is **locally valid** if

- $ph_i \equiv H(b_{i-1})$
- $h_i \equiv h_{i-1} + 1 \equiv i$
- $ts_i - ts_{i-1} \in [\Delta t - \varepsilon; \Delta t + \varepsilon]$
- $r^1_i = \gamma(r^{n_{i-1}}_{i-1}, tx^1_i)$
- $r^j_i = \gamma(r^{j-1}_i, tx^j_i) \quad \forall j \in 2, \dots, n_i$

! Δt and ε are blockchain specific parameters

Blocks & Block Chains IV

Blocks execution

A *locally-valid* block b_i can be **applied** to the system state $s_{i-1} = r_{i-1}^{n_{i-1}}$ produced by the previous block b_{i-1} by means of function Γ , producing a novel state s_i , a.k.a. the **resulting** state of b_i

$$\Gamma(s_{i-1}, b_i) = r_i^{n_i} \equiv s_i$$

by applying all transactions in b_i respecting their order, through $\gamma(\cdot, \cdot)$



Blocks & Block Chains V

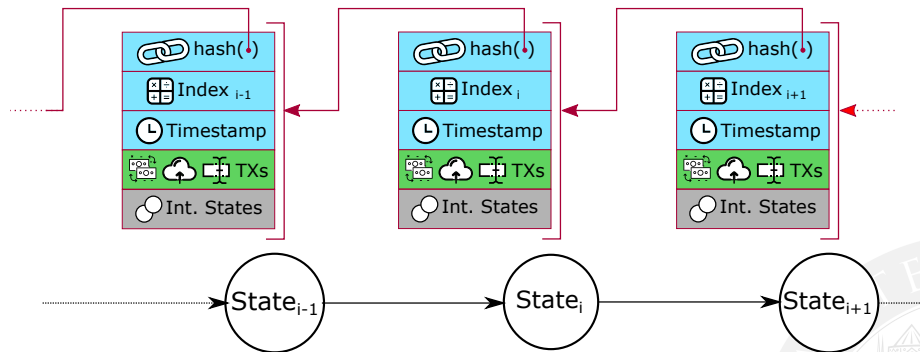


Figure: Graphical representation of the block-chain from the **global** viewpoint

! "Int. States" = "Intermediary states"

Blocks & Block Chains VI

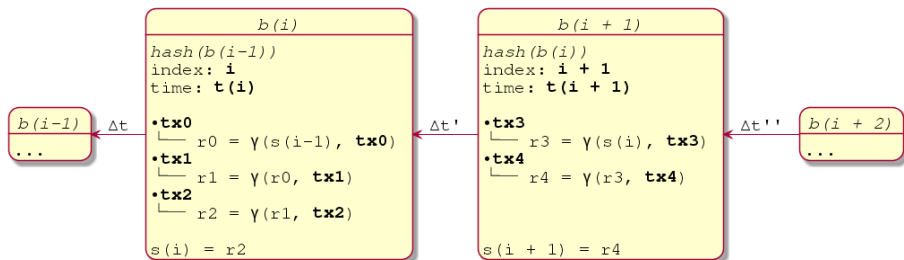
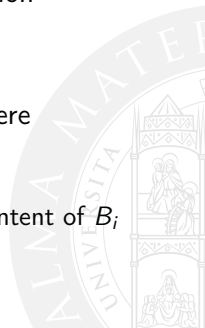


Figure: Graphical representation of blocks execution

Recap: Block Features

- replication + hash-chaining $\rightsquigarrow$ **untamperability** of the past
- hash-chain + time + ordering $\rightsquigarrow$ timestamping/notary service^[Haber and Stornetta, 1991]
- hash-chain + cryptographic signature $\rightsquigarrow$ $\begin{cases} \text{accountability} \\ \text{non-repudiation} \end{cases}$
- they are supposed to be published (almost) **periodically**
 - remember parameter Δt ?
- ! in the general case $\lim_{n \rightarrow \infty} \mathbb{P}[\text{inconsistent}(B_i)] = 0$, where
 - B_i is the i^{th} block
 - n is the amount of successor blocks of B_i
 - $\text{inconsistent}(B_i)$ is true if not all nodes agree on the content of B_i



Transaction & Block Workflow

The *genesis* block

The very first block is assumed to be **somehow shared** between the replicas

Transaction life-cycle — part I

- 0 a user U **wants** to update the system state
- 1 he/she properly **compiles** it with operation and arguments
- 2 he/she **signs** it with his/her private key K_{pr}^U
- 3 he/she **spreads** the transaction to on or more replicas

⋮

A Block's Life I

Block life-cycle

Each node, **periodically** with period Δt

- ① **listens** for transactions published by clients
- ② **validates** & **executes** them
 - ! invalid transactions are simply **dropped**
- ③ **compiles** the new local **candidate block**
- ④ **participates** to the **consensus algorithm** with other replicas
 - i.e., **negotiates** the next block with its peers

A Block's Life II

Byzantine faults may arise . . .

- because of **corrupted** replicas, or
- simply because replicas may have **perceived** TX in different **order**
! this is why we need step 4



A Block's Life III

Transaction life-cycle — part II

- ⋮
- ④ the transaction is **validated** by peers upon reception
 - and **dropped** if invalid
- ⑤ each transaction is **eventually** executed
 - producing an **intermediary state**
- ⑥ they are both included into some block
- ⑦ the block is **eventually** appended to the blockchain
 - i.e. a **consensus protocol** confirms the block
 - (there including its transactions)

! *these life cycles may vary a lot depending on the specific consensus algorithm exploited*

Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - **Consensus & Mining**
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



Consensus & Mining I

Permission-ed BCT

constrain users' IDs through **CAs**



“classical” quorum/leader-based consensus algorithms

- BFT algorithms

e.g. PBFT, [Castro and Liskov, 2002]
BFT-
SMaRt, [Sousa and Bessani, 2012]
HoneyBadger
BFT [Miller et al., 2016]

- non-BFT algorithms

e.g. Paxos, [Lamport, 1998]
Raft, [Ongaro and Ousterhout, 2014]
ZooKeeper, Google Chubby

Permission-less BCTs

open access to any (K_{pub}, K_{pr})



“novel” competition-based approaches

e.g. Proof-of-Work, [Back, 2002]
Proof-of-Stake, [Kiayias et al., 2017]
Proof-of-Elapsed-Time, [Chen et al., 2017]
IOTA Tangle [Popov et al., 2019]

Comparisons & surveys available in the literature

[Aublin et al., 2015, Cachin and Vukolić, 2017]

! CA = Certification Authority

Consensus & Mining II

Permission-ed BCTs

“classical” **quorum/leader**-based consensus algorithms

- assumptions on the amount N of replicas, and their IDs
UB up to $\sim 100 / 1000$
- high throughput in terms of TX/second
OoM ~ 1000 TX/s
- “exact” consistency
- ideal for closed multi-administrative organizations
- solves Sybil attack with CAs

Permission-less BCTs

“novel” **competition**-based approaches

- no assumption on N
UB virtually ∞
- low throughput
OoM ~ 10 TX/s
- probabilistic consistency
- ideal for open systems
eg cryptocurrencies
- solves Sybil attack with PoW

! UB = Upper Bound

! OoM = Order of Magnitude

Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - **The Sybil Attack**
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR

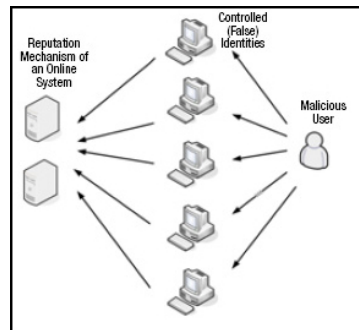


The Sybil Attack I

Distributed P2P systems/protocols where

- critical decisions are often made on a **majority** basis
- entities must **vote** in order for a majority to be established
- entities are identified within the system by means of their **IDs**
- it is easy or economically feasible for an attacker to create several **IDs**

may be subject to the **Sybil attack** [Douceur, 2002]



The Sybil Attack II

Sybil attack

An attacker subverts the P2P system by creating a large number of **pseudonymous** identities, using them to gain a disproportionately large influence within the system



Focus on...

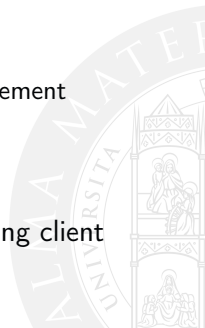
- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - **The PBFT Protocol**
 - The Proof-of-Work Mechanism
 - Blockchain Generations
- 7 Universal SMR



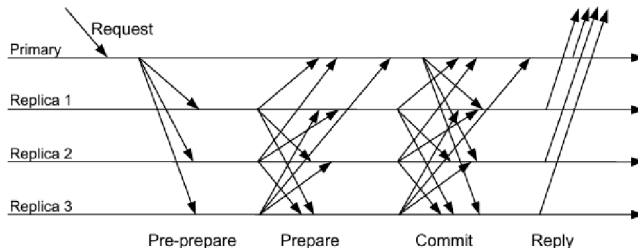
The PBFT Protocol I

The “Practical Byzantine Fault Tolerance” (PBFT) algorithm^[Castro and Liskov, 2002] is a BFT **consensus** protocol where

- a **leader** is elected among the replicas
- clients send their **requests** to the leader and wait for at least $(f + 1)$ **replies** from the replicas
 - where f = maximum amount of **faulty** replicas
- the leader **multi-casts** each request to the other replicas
 - replicas exchange several messages, **negotiating** an agreement
 - this process traverses two rounds
 - **prepare**
 - **commit**
- finally, replicas send their **responses** back to the requesting client



The PBFT Protocol II



- when the leader is expected to be faulty (e.g. timeouts) a new one must be **selected**
- this, again, implies replicas negotiating the **next leader**

Problems

- how many messages must be exchanged in case $N = 1000$?
- what could happen if **fictitious** replicas IDs could be created?

Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - **The Proof-of-Work Mechanism**
 - Blockchain Generations
- 7 Universal SMR



Proof-of-Work (PoW), a.k.a. Mining I

- PoW was originally conceived for preventing email spam^[Back, 2002]
- nowadays, it is what endows cryptocurrencies with their value
- PoW = computational puzzle + branch resolution strategy

The computational puzzle

- replicas (a.k.a. miners) compete to be the first one to solve a computational puzzle, approximately once every $\Delta t \pm \varepsilon$ seconds
- the proof of the effort is easy to verify and included into the block
 - only blocks containing the proof are considered valid
- the first one replica succeeding with the puzzle mines the next block of the blockchain
- computational difficulty must be variable and controllable

Proof-of-Work (PoW), a.k.a. Mining II

Example: Bitcoin's computational puzzle

Editing (a field of) the candidate block until its hash starts with a given amount of 0

Algorithm sketch

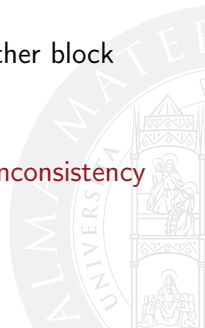
- 1 candidate block $b_i = (ph_i, h_i, ts_i, m_i, \bar{tx}_i)$, where $m_i = 0$
- 2 while $H(b_i) \geq h_{threshold}$: increase m_i by 1

Example: Ethereum's computational puzzle

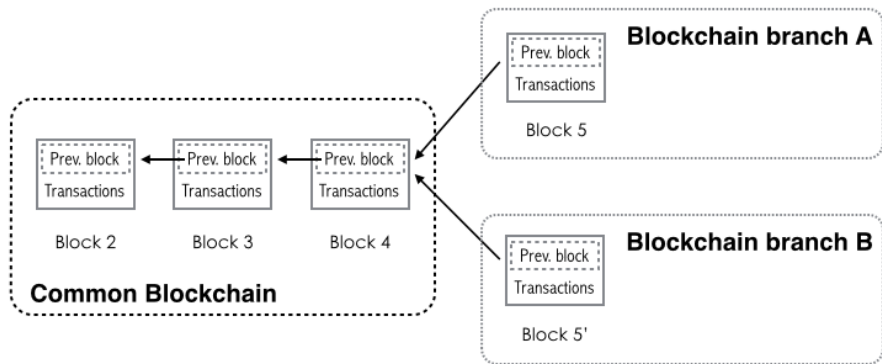
- a ~ 1 GiB pseudo-random graph is generated in a reproducible way
- candidate block's content used to select some parts of the graph
- such parts are hashed together and included into the block

Proof-of-Work (PoW), a.k.a. Mining III

- mined blocks are **published** over the P2P network of replicas
 - but propagation on the network **requires time**
- other honest replicas **accept** as soon as they hear of it
 - they **recognise** they have lost the round
 - they start **mining** on top of the new block
- simultaneously, some other replica may have **mined** another block
 - maybe because it has not still received the other block
 - maybe because it is **corrupt** or malicious
- this situation is known as a **branch** and it represent an **inconsistency**



Proof-of-Work (PoW), a.k.a. Mining IV



- **branches** represent many possible **versions** of the transactions ledger

Proof-of-Work (PoW), a.k.a. Mining V

The branch resolution strategy

- *branches are unavoidable*
- a **local** rule must be exploited by replicas to decide which branch should be **considered** the true one
- as long the **majority** of replicas follow the rule, a single branch should **eventually** be selected by all replicas $\implies$ *eventual consistency*

Example: Bitcoin's branch resolution strategy

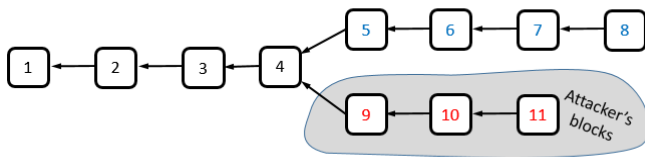
Longest chain rule

Example: Ethereum's branch resolution strategy

Greedy Heaviest Observed SubTree (GHOST^[Sompolinsky and Zohar, 2013])

Longest Chain Rule I

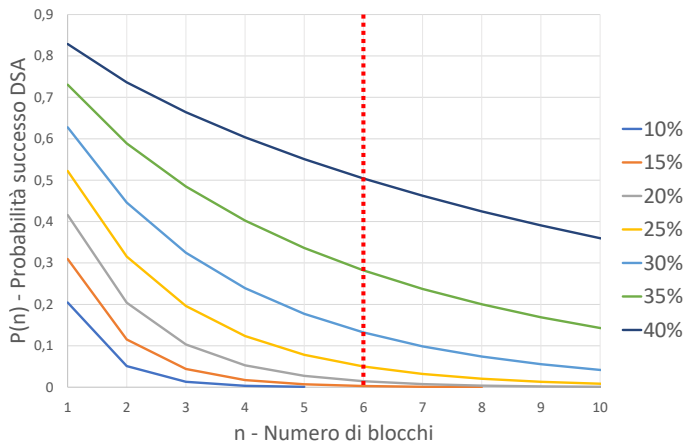
- honest replicas must always consider the **longest** chain they heard about first
- even if a branching occurs, the two branches will not evolve at the **exact same speed**
- a branch will **eventually become longer**, more quickly than the other
 - thus, eventually, the other branch will be abandoned (becoming **orphan**)
- we **cannot prevent** corrupt replicas from generating branches
 - but how **likely** is it a single replica produces a **longer branch in less time** than the others?



Longest Chain Rule II

$$r = \frac{\text{adversary}_{CP}}{\text{honest}_{CP}}$$

$$\mathbb{P}[n \mid r] = 1 - \sum_{k=0}^n \frac{(nr)^k e^{-nr}}{k!} (1 - r^{n-k}) \quad [\text{Nakamoto, 2008}]$$



In Bitcoin

- $n_{\text{threshold}} = 6$
- $\approx 1h$ since $\mathbb{E}[\Delta T] = 10m$
- 99.999% secure if $\text{adversary}_{CP} < 13\% \text{ total}_{CP}$

PoW Interesting Features

- competition-based, local, eventually-consistent, stochastic approach
- self-adaptive mining difficulty, s.t. $\mathbb{E}[\Delta t] = \text{const}$
 - updates compensate the computing power variation of the replicas
 - e.g., in Bitcoin $h_{\text{threshold}}$ is periodically updated
 - e.g., Ethereum comes with similar mechanism + ASIC-resistance
- only computing power (CP) matters here
 - Sybil-attack resistant
 - CP distribution & majority rule (51% attack)^[Eyal and Sirer, 2014]
- notice that now the invariant $\mathbb{E}[\Delta t] = \text{const}$ implicitly states ...
 - ... the system maximum update frequency is $1/\mathbb{E}[\Delta t]$



PoW: Incentives & Disincentives

HP: miners are *rational*, i.e. selfish

Why should miners honestly participate the PoW-based consensus?

- likely, those are not even their own transactions
- PoW is **costly** in terms of resources, and, therefore, **real money**

Economic incentives

Miners “winning” the computational puzzle are rewarded with money

- such money is created *ex-nihilo* by the protocol
- it is an incentive for fairly competing & **well-behaving**

Economic *disincentives*

Symmetrically, **attacks** such as deliberate branching are not strictly prevented but they are made **extremely costly**

PoW, Cryptocurrencies & Security

Within cryptocurrencies-based blockchains ...

PoW, economical value, and security are **strictly** related

- 1 PoW endows the cryptocurrency with its **economical value**
- 2 miners require **economical compensation** for their job
- 3 the miners' job is the **gasoline** of a blockchain system
- 4 miners will behave honestly as long as it is **convenient** to do so
- 5 it is convenient as long as the cryptocurrency has value



Focus on...

- 3 Expectations
- 4 Background
 - Cryptographic Tools
 - Schemes
 - Applications
- 5 State Machine Replication
 - Overview
 - Deterministic Stateful Computation
 - Issues of Open Distributed Systems
 - CAP Theorem
 - Byzantine Fault Tolerance
- 6 Blockchain Main Elements
 - User Identifiers
 - The World State
 - Transactions
 - Blocks & Block Chains
 - Consensus & Mining
 - The Sybil Attack
 - The PBFT Protocol
 - The Proof-of-Work Mechanism
 - **Blockchain Generations**
- 7 Universal SMR

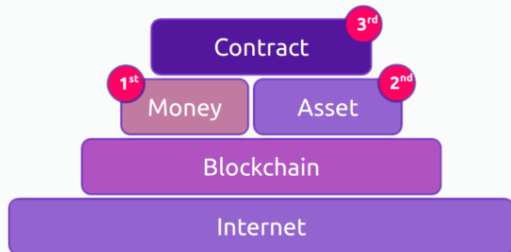


Three Blockchain Generations I

Popular classification of BCT [Swan, 2015]

- three BCT generations

- 1 cryptocurrencies
- 2 custom assets
- 3 smart contracts



! the model presented so far essentially includes generations 1 and 2

Three Blockchain Generations II

Several expected applications

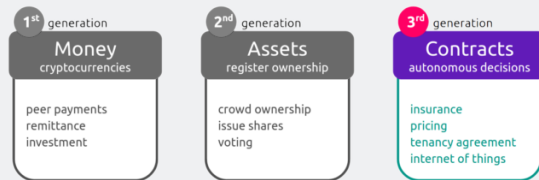


Interesting applications for generations 1-2

- payments for online purchases, in place of PayPal
- smart property
- unified medical records
- non-censurable social/news
- shared balances among companies
- public tenders
- naming systems & **access control**
- voting systems

Towards Smart Contracts

What about generation 3?



- so far, we have described a **trustable** way for storing/updating data without **single-points-of-trust**
- how can we trust the users **producing** that data?
- we could automatise asset-management workflows via **software** ...
 - we must still **trust** developers and organizations producing the software
- we need **trustable** computational entities (i.e. **agents**) able to automatically manage assets
 - ⇒ i.e., **smart contracts**

Next in Line...

- 3 Expectations
- 4 Background
- 5 State Machine Replication
- 6 Blockchain Main Elements
- 7 Universal SMR**



“Universal” State Machine Replication

$$UTM : TM = \text{Interpreter} : \text{Program} = ??? : SMR$$

! UTM $\stackrel{\text{def}}{=}$ Universal Turing Machine | TM $\stackrel{\text{def}}{=}$ Turing Machine

- we could replicate a stateful deterministic program implementing a particular business logic ! e.g. a bank ledger
- in the exact same way, we could replicate a deterministic program implementing an **interpreter** ! $\approx$ a program which executes other programs
 - i.e., what we call “**Universal** State Machine Replication” (USMR)
- the **admissible operations** on such a *replicated* interpreter should enable users **deploy**, **undeploy**, or **invoke** programs

SC-enabled blockchains

Smart-contract-enabled blockchains essentially act as replicated “universal” state machines on which **smart contracts** can be deployed

Part III

Smart Contracts



Next in Line...

8 Smart Contracts

9 Examples & Use Cases

10 Issues



Focus on...

8 Smart Contracts

- **Definition**
- Features & Expectations
- Run Time
- Updating the BCT Model
- Expressiveness vs. Termination

9 Examples & Use Cases

- An Example: Solidity
- Use Cases

10 Issues



Smart Contracts (SC)^[Szabo, 1997]

Smart contract

Smart contracts (SC) are *stateful*, *user-defined*, *reactive*, *immutable* – therefore **trustable** –, deterministic processes executing *replicated* and *expressive* computations on the blockchain

stateful — they **encapsulate** their own state, like objects in OOP

reactive — they can only be triggered by some **off-chain** client

user-defined — users can **deploy** their own smart contracts implementing **any** business logic

immutable — their code **cannot be altered** after deployment

replicated — the blockchain acts as a replicated **interpreter** for such processes

expressive — they are expressed with a **Turing-complete** language

Focus on...

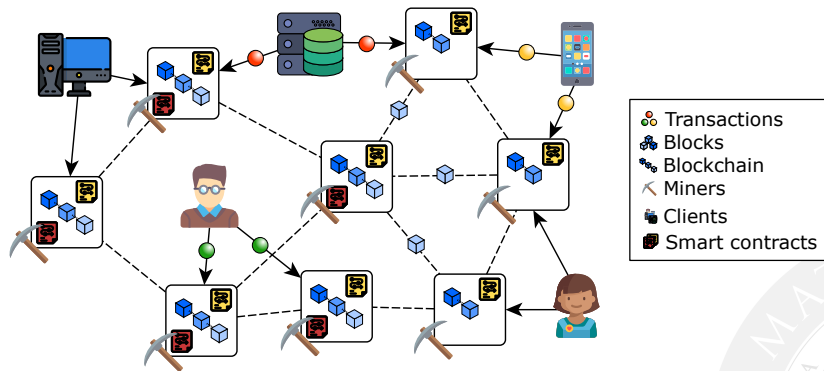
- 8 Smart Contracts
 - Definition
 - **Features & Expectations**
 - Run Time
 - Updating the BCT Model
 - Expressiveness vs. Termination
- 9 Examples & Use Cases
 - An Example: Solidity
 - Use Cases
- 10 Issues



Features of Smart Contracts

- SC can be used to **automate** workflows involving **two or more parties**
 - they can act as **intermediaries** between the parties
 - reducing failures due to **corrupted/selfish** *human* operators
 - making workflow (more) trustable
- immutability + inspectability + accountability + replication
⇒ can be **trusted** in handling critical assets (e.g. **money**)
- the *code is always right*, the true history is on the blockchain
 - reducing disputes
 - removing the need for arbitration
- lack of situatedness: totally **disembodied** ^[Mariani and Omicini, 2013] data & computation
 - like the cloud, but with no single point of trust
- at any moment, ***N* copies** exist of a given SC, one for each **replica**
 - but they always compute **as one**
 - “smart contract instance” = “the many copies of a SC instance”

Smart Contract Instances



- SC instances are cloned too among the servers replicas
- in the picture, different colours represents different identities

Focus on...

- 8 Smart Contracts
 - Definition
 - Features & Expectations
 - **Run Time**
 - Updating the BCT Model
 - Expressiveness vs. Termination
- 9 Examples & Use Cases
 - An Example: Solidity
 - Use Cases
- 10 Issues



Smart Contract Operations

Usually, at least two **operations** are provided to clients

deploy — accepts the SC code and has the effect of instantiating it on all the replicas, generating a unique **identifier/address** for the SC

invoke — accepts a **SC identifier** and a number of **arguments** and it has the effect of executing *that* SC with *those* arguments

A **reactive computational model**



Smart Contract Deployment

- ① initially there are no smart contracts
- ① a user can instantiate a smart contract by publishing its source/byte-code within a **deployment TX**
 - the TX also initialises the SC state
 - the protocol assigns an **univocal address** to the new SC
- ② once the transaction is confirmed, you can assume the an SC instance is running on **all** replicas
 - no a huge effort: it is just **listening** for invocations



Smart Contract Invocation

- ➊ users can trigger a SC by publishing an **invocation TX**
 - specifying the SC address as recipient
 - providing some input data specifying the requested computation
- ➋ eventually, each replica will **receive** the TX and **execute** it
 - the SC code usually **specifies** what to do with the provided arguments
- ➌ if the computation terminates **without exceptions**, any produced side effects (on the SC state) become part of the new **intermediate system state**
 - otherwise, side effects are simply **dropped**, thus the new intermediate state **coincides** with the previous one
- ➍ the wrapping block is eventually **confirmed** by the consensus protocol
 - and the invoked computation can be **actually considered** executed

Focus on...

- 8 Smart Contracts
 - Definition
 - Features & Expectations
 - Run Time
 - **Updating the BCT Model**
 - Expressiveness vs. Termination
- 9 Examples & Use Cases
 - An Example: Solidity
 - Use Cases
- 10 Issues



Updating the BCT Model I

Entities can either be Users or Smart Contracts

$$\langle \text{EntityID} \rangle ::= \langle \text{UserID} \rangle \mid \langle \text{SmartContractID} \rangle$$

The System State is still a set of assets

$$\begin{aligned} \langle \text{SystemState} \rangle &::= \langle \text{Asset} \rangle \\ &\mid \langle \text{SystemState} \rangle \cup \langle \text{SystemState} \rangle \end{aligned}$$

Assets can either be User data or Smart Contract states

$$\begin{aligned} \langle \text{Asset} \rangle &::= \langle \text{UserID} \rangle, \langle \text{Data} \rangle \\ &\mid \langle \text{SmartContractID} \rangle, \langle \text{SmartContractState} \rangle \end{aligned}$$

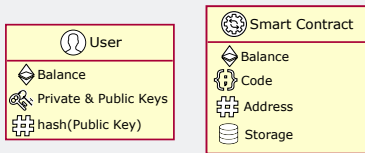
Updating the BCT Model II

Smart contracts *states* are made of *storage* + *code*

$$\langle \text{SmartContractState} \rangle ::= \langle \text{Storage} \rangle, \langle \text{Code} \rangle$$

- think about some SC *storage* as an object **fields** in OOP
- think about some SC *code* as an object **methods** in OOP
- a SC code can **alter** its storage, or change its behaviour based on it

In a nutshell



Updating the BCT Model III

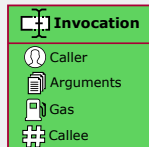
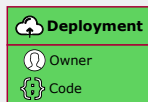
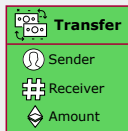
Transactions can be *deployments*, *invocations*, or *transfers*

$$\langle TX \rangle ::= \langle EntityID \rangle, \langle Operation \rangle, \langle Signature \rangle$$

$$\begin{aligned} \langle Operation \rangle ::= & \text{deploy}, \langle Code \rangle \\ & | \text{invoke}, \langle Arguments \rangle, \langle SmartContractID \rangle \\ & | \text{transfer}, \langle EntityID \rangle, \langle Value \rangle \end{aligned}$$

In a nutshell

! the “gas” field will be presented in a few slides



Updating the BCT Model IV

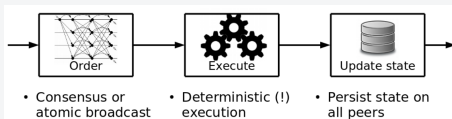
The γ function executes TX as follows

- creating a **new** SC in case of a **deploy** TX
- updates the invoked SC **state** in case of a **invoke** TX
- updates both the sender's & recipient's **balance** in case of a **transfer** TX

The γ function is *unchanged*

i.e., it still executed a block's TX **sequentially**

We just described the *Order-Execute* architecture [Androulaki et al., 2018]



Focus on...

- 8 Smart Contracts
 - Definition
 - Features & Expectations
 - Run Time
 - Updating the BCT Model
 - **Expressiveness vs. Termination**
- 9 Examples & Use Cases
 - An Example: Solidity
 - Use Cases
- 10 Issues



Issues Arising from Turing-completeness I

What would be the effect of invoking a **non-terminating program** as a SC?

- impossible to filter out non-terminating programs
 - based on the **halting problem**
- in open distributed systems, users cannot be assumed to simply **well behave**

⇒ need to **prevent/discourage** users from deploying/invoking **infinite or long** computations

How to address this issue

BCT address this issue in different ways

eg Ethereum $\longleftrightarrow$ Gas

eg HLF $\longleftrightarrow$ **Execute-Order-Validate** architecture

Ethereum and Gas I

Ethereum proposes **gas**, i.e., making users pay for computation executions

- TX are endowed with two more fields: **gasLimit** & **gasPrice**
 - miners could delay TX having a low **gasPrice**
 - users can increase their priority by increasing **gasPrice**
- upon execution, each bytecode instruction increases the **g** counter
 - according to a **price list**^[Wood, 2014]
- whenever $g > \text{gasLimit}$ an exception is raised, reverting side effects
- in any case, upon termination, the issuer balance is decreased of $\Delta ETH = \text{gasPrice} \cdot g$
 - the winning miner can redeem ΔETH as a compensation for its computational effort

Ethereum and Gas II

Practical consequence

The **economical** dimension of computation has to be taken into account when designing Ethereum smart contracts

Theoretical consequence

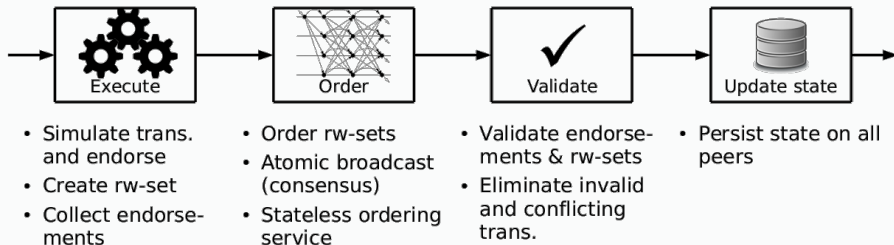
SC are not really Turing-complete

- *quasi*-Turing-completeness^[Wood, 2014]



HLF & EOv Architecture [Androulaki et al., 2018] |

Execute-Order-Validate (EOV) architecture in a nutshell



HLF & EOV Architecture [Androulaki et al., 2018] II

Main features

- three sorts of nodes in the BCT system
 - peers** (i.e. clients) — issuing TX
 - endorsers** — executing TX (and therefore SC)
 - orderers** — ordering TX via consensus
- ordering occurs *after* execution, only if a *sufficient* amount of endorsers accept the transaction
 - acceptance involves validation, consistency & execution
- long-lasting / non-terminating TX do not slow-down orderers
 - just endorsers. . .
 - . . . which may deny their endorsement in case of long-lasting / non-terminating computations

Next in Line...

8 Smart Contracts

9 Examples & Use Cases

10 Issues



Focus on...

- 8 Smart Contracts
 - Definition
 - Features & Expectations
 - Run Time
 - Updating the BCT Model
 - Expressiveness vs. Termination
- 9 Examples & Use Cases
 - **An Example: Solidity**
 - Use Cases
- 10 Issues



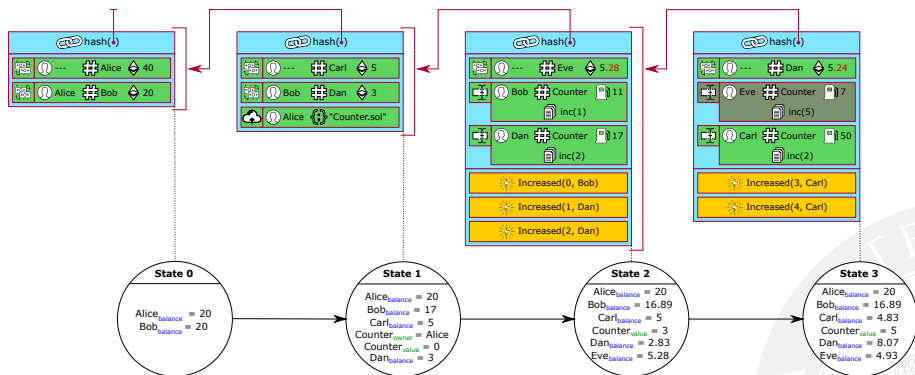
Ethereum SC: An Example with Solidity I

- Solidity is an object-oriented, high-level language for implementing smart contracts
- <http://solidity.readthedocs.io>

Listing 1: An Ethereum SC with Solidity

```
contract Counter {  
  
    event Increased(uint oldValue, address cause); // event definition  
  
    address owner;  
    uint value;  
  
    function Counter() public { owner = msg.sender; } // <-- constructor  
  
    function inc(uint times) public { // <-- API  
        for (uint i = 0; i < times; i++) {  
            emit Increased(value++, msg.sender);  
        }  
    }  
}
```

Ethereum SC: An Example with Solidity II



Focus on...

- 8 Smart Contracts
 - Definition
 - Features & Expectations
 - Run Time
 - Updating the BCT Model
 - Expressiveness vs. Termination
- 9 Examples & Use Cases
 - An Example: Solidity
 - **Use Cases**
- 10 Issues



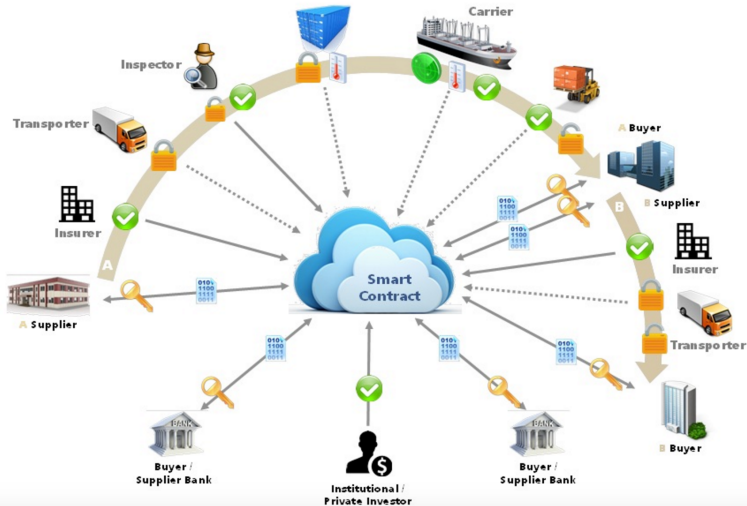
Use Cases & Expectations

Smart contracts are expected to act as **intermediaries** in

- automatic money management or payments
 - e.g., bills, insurances, shares distribution
- shared governance
 - e.g. Distributed Autonomous Organizations (DAOs)
- automatic asset/goods/**supply-chain** tracking
 - e.g. in combination with **IoT**



Smart Contracts + Internet of Things (IoT) I



Smart Contracts + Internet of Things (IoT) II

- ❶ the possibly-many **movements** of goods could be automatically tracked
 - by one or more **interacting** smart contracts
- ❷ departures and arrivals of goods may be detected by **IoT sensors**
 - no way for the many involved actors to lie about **quantities**, or **timings**
- ❸ SC could automatically perform **payments**
 - as soon as movements are detected
 - or **refunds** in case they are not
- ❹ founders, creditors, inspectors, and **end-users** can always **know** the distribution path of products
 - they just have to **agree** on the **policy** enforced by the SCs code

The main idea

Delegating trust-critical decisions to transparent, and trustable SW agents

Next in Line...

- 8 Smart Contracts
- 9 Examples & Use Cases
- 10 Issues**



Early Technology

Smart contracts are quite an immature technology

- no privacy or secrets
- no randomness
- inter-SC communication & re-entrancy
- immutability
- control flow & pure reactivity
- disembodiment & lack of concurrency
- granularity of the computational cost

Neither Privacy nor Secretness

Notice that...

... every information ever published on the blockchain is available on the blockchain

- the private state of a smart contract is **not secret**
- pseudo-anonymity can be broken with *statistics* or **machine learning**
- illegal/anti-ethic behaviour can be revealed **years later**
- ? no secret voting?



Poor Randomness

Notice that...

... it is difficult to achieve (pseudo-)randomness because of the lack of **non-determinism**

- real randomness cannot be exploited (replicas would diverge)
- most of the blocks observable information is **manipulable** by miners
e.g. timestamp, height, hash, etc.
- no trivial way to find a **non-manipulable** random seed
- ? no lottery?



Smart Contract Inter-communication

Notice that...

... SC are essentially **objects** communicating by means of synchronous **method calls**

- the control flow originating from a user may traverse more than one SC
- the caller waits for the callee
- *non-reentrant* SC code could fail if invoked back by an invoked SC^[Atzei et al., 2017, Luu et al., 2016]
- and it may lead to undesired behaviours, as well as **frauds**^[Dupont, 2017]
- ? is OOP the best programming paradigm?

No Way to Fix Bugs

Notice that...

... SC code is **immutable**. Immutability is both a blessing and a curse.

- buggy/fraudulent contracts cannot be fixed, updated, replaced, or un-deployed
 - they will remain so, wasting miners resources
- paramount importance of correct-design and formal validation
 - a problem *per se* in Turing-complete languages



Lack of Pro-activeness

Notice that...

... SC are purely reactive computational entities

- they always need to borrow some user's control flow
- they are time-aware but **not time-reactive**
- ! they cannot **schedule or postpone** computations
 - no periodic computation (e.g. payment)



Disembodiment & Lack of Concurrency

Notice that...

... computation is logically located **everywhere** and execution is strictly **sequential**

- independent computation *cannot* be executed **concurrently**
- computations only making sense **locally** need to be replicated **globally**
- ! smart contracts cannot be used to **parallelise** computations



Granularity of Computation-related Costs

Notice that. . .

. . . Ethereum is not the first platform applying a **price** to computation:
e.g. common practice on the Cloud, and the **X-as-a-Service** paradigm

BTW, is the instruction-level **cost granularity** the best one?

e.g. when implementing the **publish-subscribe** mechanism, the publisher
pays the higher price^[Ciatto et al., 2018]



Distributed Ledger Technology

Blockchain as Middleware

Distributed Systems / Case Study

Andrea Omicini Giovanni Ciatto

<mailto:andrea.omicini@unibo.it> <mailto:giovanni.ciatto@unibo.it>

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



References I

- [Androulaki et al., 2018] Androulaki, E., Barger, A., Bortnikov, V., Cachin, C., Christidis, K., De Caro, A., Enyeart, D., Ferris, C., Laventman, G., Manevich, Y., Muralidharan, S., Murthy, C., Nguyen, B., Sethi, M., Singh, G., Smith, K., Sorniotti, A., Stathakopoulou, C., Vukolić, M., Cocco, S. W., and Yellick, J. (2018).
Hyperledger Fabric: A distributed operating system for permissioned blockchains.
 In *13th EuroSys Conference (EuroSys'18)*, New York, NY, USA. ACM
 DOI:10.1145/3190508.3190538
- [Atzei et al., 2017] Atzei, N., Bartoletti, M., and Cimoli, T. (2017).
A survey of attacks on Ethereum smart contracts (SoK).
 In *Principles of Security and Trust*, volume 10204 of *LNCs*, pages 164–186. Springer
 DOI:10.1007/978-3-662-54455-6_8
- [Aublin et al., 2015] Aublin, P.-L., Guerraoui, R., Knežević, N., Quéma, V., and Vukolić, M. (2015).
The next 700 BFT protocols.
ACM Transactions on Computer Systems, 32(4):1–45
 DOI:10.1145/2658994
- [Back, 2002] Back, A. (2002).
Hashcash - a denial of service counter-measure
<http://www.hashcash.org/hashcash.pdf>
- [Brewer, 2000] Brewer, E. A. (2000).
Towards robust distributed systems (abstract).
 In *19th Annual ACM Symposium on Principles of Distributed Computing (PODC '00)*, page 7, New York, New York, USA. ACM Press
 DOI:10.1145/343477.343502



References II

- [Cachin and Vukolić, 2017] Cachin, C. and Vukolić, M. (2017).
Blockchain consensus protocols in the wild (keynote talk).
In *31st International Symposium on Distributed Computing (DISC 2017)*, volume 91 of *LIPICs*, pages 1:1–1:16,
Dagstuhl, Germany
DOI:10.4230/LIPICs.DISC.2017.1
- [Castro and Liskov, 2002] Castro, M. and Liskov, B. (2002).
Practical byzantine fault tolerance and proactive recovery.
ACM Transactions on Computer Systems, 20(4):398–461
DOI:10.1145/571637.571640
- [Charron-Bost et al., 2010] Charron-Bost, B., Pedone, F., and Schiper, A., editors (2010).
Replication. Theory and Practice, volume 5959 of *Lecture Notes in Computer Science*.
Springer, Berlin, Heidelberg
DOI:10.1007/978-3-642-11294-2
- [Chen et al., 2017] Chen, L., Xu, L., Shah, N., Gao, Z., Lu, Y., and Shi, W. (2017).
On security analysis of proof-of-elapsed-time (PoET).
In Spirakis, P. and Tsigas, P., editors, *Stabilization, Safety, and Security of Distributed Systems (SSS 2017)*,
volume 10616 of *Lecture Notes in Computer Science*, pages 282–297. Springer, Cham
DOI:10.1007/978-3-319-69084-1_19
- [Ciatto et al., 2018] Ciatto, G., Mariani, S., and Omicini, A. (2018).
Blockchain for trustworthy coordination: A first study with Linda and Ethereum.
In *2018 IEEE/WIC/ACM International Conference on Web Intelligence (WI)*, pages 696–703
DOI:10.1109/WI.2018.000-9



References III

[Douceur, 2002] Douceur, J. (2002).

The Sybil attack.

In *1st International Workshop on Peer-to-Peer Systems (IPTPS '01)*, pages 251–260.

Revised Papers

<https://dl.acm.org/doi/10.5555/646334.687813>

[Dupont, 2017] Dupont, Q. (2017).

Experiments in algorithmic governance. a history and ethnography of “The DAO,” a failed decentralized autonomous organization.

In *Bitcoin and Beyond. Cryptocurrencies, Blockchains, and Global Governance*, chapter 8. Routledge, London, UK, 1st edition

DOI:10.4324/9781315211909-8

[Eyal and Sirer, 2014] Eyal, I. and Sirer, E. G. (2014).

Majority is not enough: Bitcoin mining is vulnerable.

In Christin, N. and Safavi-Naini, R., editors, *Financial Cryptography and Data Security (FC 2014)*, volume 8437 of *Lecture Notes in Computer Science*, pages 436–454. Springer

DOI:10.1007/978-3-662-45472-5_28

[FG DLT, 2019a] FG DLT (2019a).

DLT reference architecture.

ITU Technical Specification FG DLT D3.1, International Telecommunication Union

<https://www.itu.int/en/ITU-T/focusgroups/dlt/Documents/d31.pdf>

[FG DLT, 2019b] FG DLT (2019b).

DLT terms and definitions.

ITU Technical Specification FG DLT D1.1, International Telecommunication Union

<https://www.itu.int/en/ITU-T/focusgroups/dlt/Documents/d11.pdf>



References IV

[FG DLT, 2019c] FG DLT (2019c).

DLT use cases.

ITU Technical Report FG DLT D2.1, International Telecommunication Union

<https://www.itu.int/en/ITU-T/focusgroups/dlt/Documents/d21.pdf>

[Fischer et al., 1985] Fischer, M. J., Lynch, N. A., and Paterson, M. S. (1985).

Impossibility of distributed consensus with one faulty process.

Journal of the ACM (JACM), 32(2):374–382

DOI:10.1145/3149.214121

[Focus Group on Application of Distributed Ledger Technology, 2017] Focus Group on Application of Distributed Ledger Technology (2017).

FG DLT home page

<https://www.itu.int/en/ITU-T/focusgroups/dlt/>

[Haber and Stornetta, 1991] Haber, S. and Stornetta, W. S. (1991).

How to time-stamp a digital document.

Journal of Cryptology, 3(2):99–111

DOI:10.1007/BF00196791

[ISO/TC 307, 2020] ISO/TC 307 (2020).

Blockchain and distributed ledger technologies — vocabulary.

ISO 22739:2020, International Organization for Standardization

<https://www.iso.org/standard/73771.html>



References V

[ISO/TC 307, 2024] ISO/TC 307 (2024).

Blockchain and distributed ledger technologies — vocabulary.

ISO 22739:2024, International Organization for Standardization

<https://www.iso.org/standard/82208.html>

[Kiayias et al., 2017] Kiayias, A., Russell, A., David, B., and Oliynykov, R. (2017).

Ouroboros: A provably secure proof-of-stake blockchain protocol.

In Katz, J. and Shacham, H., editors, *Advances in Cryptology – CRYPTO 2017*, volume 10401 of *Lecture Notes in Computer Science*, pages 357–388. Springer

DOI:10.1007/978-3-319-63688-7_12

[Lamport, 1998] Lamport, L. (1998).

The part-time parliament.

ACM Transactions on Computer Systems, 16(2):133–169

DOI:10.1145/279227.279229

[Lamport et al., 1982] Lamport, L., Shostak, R., and Pease, M. (1982).

The Byzantine generals problem.

ACM Transactions on Programming Languages and Systems, 4(3):382–401

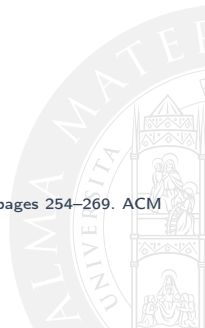
DOI:10.1145/357172.357176

[Luu et al., 2016] Luu, L., Chu, D.-H., Olickel, H., Saxena, P., and Hobor, A. (2016).

Making smart contracts smarter.

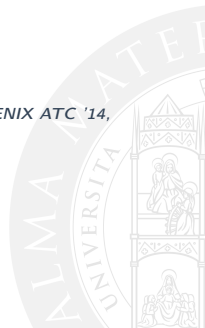
In *2016 ACM SIGSAC Conference on Computer and Communications Security (CCS'16)*, pages 254–269. ACM Press

DOI:10.1145/2976749.2978309



References VI

- [Mariani and Omicini, 2013] Mariani, S. and Omicini, A. (2013).
 TuCSoN on cloud: An event-driven architecture for embodied / disembodied coordination.
 In Aversa, R., Kolodziej, J., Zhang, J., Amato, F., and Fortino, G., editors, *Algorithms and Architectures for Parallel Processing*, volume 8286 of *LNCS*, pages 285–294. Springer International Publishing Switzerland.
 13th International Conference (ICA3PP-2013), Vietri sul Mare, Italy, 18-20 December 2013. Proceedings, Part II
 DOI:10.1007/978-3-319-03889-6_33
- [Miller et al., 2016] Miller, A., Xia, Y., Croman, K., Shi, E., and Song, D. (2016).
 The honey badger of BFT protocols.
 In *2016 ACM SIGSAC Conference on Computer and Communications Security (CCS'16)*, pages 31–42
 DOI:10.1145/2976749.2978399
- [Nakamoto, 2008] Nakamoto, S. (2008).
 Bitcoin: A peer-to-peer electronic cash system
<http://bitcoin.org/bitcoin.pdf>
- [Ongaro and Ousterhout, 2014] Ongaro, D. and Ousterhout, J. K. (2014).
 In search of an understandable consensus algorithm.
 In Gibson, G. and Zeldovich, N., editors, *2014 USENIX Annual Technical Conference, USENIX ATC '14, Philadelphia, PA, USA, June 19-20, 2014*, pages 305–319. USENIX Association
<https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [Popov et al., 2019] Popov, S., Saa, O., and Finardi, P. (2019).
 Equilibria in the tangle.
Computers & Industrial Engineering, 136:160–172
 DOI:10.1016/j.cie.2019.07.025



References VII

- [Schneider, 1990] Schneider, F. B. (1990).
Implementing fault-tolerant services using the state machine approach: A tutorial.
ACM Computing Surveys, 22(4):299–319
DOI:10.1145/98163.98167
- [Sompolinsky and Zohar, 2013] Sompolinsky, Y. and Zohar, A. (2013).
Accelerating Bitcoin's transaction processing. fast money grows on trees, not chains.
ePrint Archive 2013/881, IACR Cryptology
<http://eprint.iacr.org/2013/881>
- [Sousa and Bessani, 2012] Sousa, J. and Bessani, A. (2012).
From Byzantine consensus to BFT state machine replication: A latency-optimal transformation.
In *9th European Dependable Computing Conference (EDCC 2012)*, pages 37–48
DOI:10.1109/EDCC.2012.32
- [Swan, 2015] Swan, M. (2015).
Blockchain: Blueprint for a New Economy.
O'Reilly
<http://www.oreilly.com/library/view/blockchain/9781491920480/>
- [Szabo, 1997] Szabo, N. (1997).
Formalizing and securing relationships on public networks.
First Monday, 2(9)
DOI:10.5210/fm.v2i9.548



References VIII

- [Tanenbaum and van Steen, 2017] Tanenbaum, A. S. and van Steen, M. (2017). *Distributed Systems. Principles and Paradigms*. Pearson Prentice Hall, 3rd edition
<https://www.distributed-systems.net/index.php/books/ds3/>
- [Wegner and Goldin, 1999] Wegner, P. and Goldin, D. (1999). *Mathematical models of interactive computing*. Technical report, Brown University, Providence, RI, USA (APICe) <https://dl.acm.org/doi/book/10.5555/864493>
- [Wood, 2014] Wood, G. (2014). *Ethereum: a secure decentralised generalised transaction ledger*
<http://ethereum.github.io/yellowpaper/paper.pdf>

